Quantum 0/1 Knapsack Solver

Software Documentation — Tier-1 (Grover / BBHT / Dürr–Høyer)

OpenModelica → Qiskit C API → Quantum Search

M. Myers
mmyers@mmyerslaw.com

2026-07-12, with the October 2026 preface (Document version 1.7)

Abstract

This document is the engineering reference for a quantum 0/1-knapsack solver written in C against the Qiskit C API and callable from OpenModelica. The subset *selection* is performed by Grover amplitude amplification wrapped in a Boyer–Brassard–Høyer–Tapp (BBHT) iteration schedule and a Dürr–Høyer maximum-finding loop. The classical exhaustive search is retained only as an independent ground-truth checker. The solver runs on a from-scratch, pure-C statevector simulator for *validation*: **no quantum speedup is claimed** (exact 0/1 knapsack is NP-hard, and the relevant cost metric is the Grover oracle-call count, which the code reports). This document covers the requirements, the fundamental publications, the architecture, the code structure, the algorithms, the implementation details, and the verification results for **Tier-1**, which is complete. Later tiers (noise modelling, hardware execution, publication-grade circuit drawing) are scoped at the end and will be added as chapters.

Contents

Read first: what changed since version 1.6 **2**

1 Introduction and Scope **4**

2 Requirements **4**
2.1 Functional requirements . . . 4
2.2 Non-functional requirements and contracts . . . 5
2.3 Correctness and honesty rules (hard requirements) . . . 5
2.4 Updated requirement (logged 2026-06-05) . . . 5

3 Fundamental Publications **5**

4 Architecture **6**
4.1 Layering and the execution seam . . . 7
4.2 The role of the Qiskit C API . . . 7
4.3 Pure-quantum scope gate . . . 7

5 Code Structure **7**

6 Algorithms **8**
6.1 The quantum predicate . . . 8
6.2 Reversible arithmetic: controlled constant-adder (§2.5) . . . 8
6.3 Comparator via two's complement (§2.6) . . . 8

6.4 Grover oracle: compute → mark → uncompute (§2.7) . . . 9
6.5 Grover diffuser (§2.8) . . . 9
6.6 Inner search: BBHT (§2.9) . . . 9
6.7 Outer loop: Dürr–Høyer maximum finding (§2.9) . . . 10
6.8 Note on computational complexity . . . 10

7 Implementation Details **10**
7.1 Pure-C statevector simulator (§1) . . . 10
7.2 Qubit layout . . . 11
7.3 Solver wiring (§3) . . . 11
7.4 OpenModelica entry (§5) and status codes . . . 11
7.5 Compile-time knobs . . . 11

8 Verification and Test Results **11**
8.1 Strategy . . . 11
8.2 Results . . . 11

9 Build and Run **12**

10 Tier-2: Portable Export and Noise Characterization **12**
10.1 OpenQASM-3 exporter (§4.5) . . . 13
10.2 Aer depolarizing noise study . . . 13
10.3 Noise-study figure . . . 14
10.4 Publication circuit drawing (`QDrawDetail`, §4.6) . . . 14
10.5 Hardware path (path-preserving) . . . 14

11 Extension Points and Future Tiers **14**

A Verified Qiskit C API gate names **15**

B Circuit schematic (current, light) **15**

Read first: what changed since version 1.6 (October 2026)

This document describes the solver as it stood in July 2026. The algorithms, the qubit layout, the test strategy and the results reported below are unchanged, but the interface to OpenModelica and several limits are not. The code in the repository is authoritative; this section lists every difference a reader will meet.

Note

Entry point. `callKnapsack` now takes **thirteen** arguments (this document's text and Table 2 show the original seven):

```
void callKnapsack(int Npatents, const double *Values, const double *Costs,
                  double MaxCost, int *PatentIndices, int *ExStatus,
                  int QDrawDetail, int SolverPath, int NStates,
                  double *PSubset, int *OracleCalls, int *Shots, int *Kind);
```

Still `void` with every output by pointer. New inputs: `SolverPath` (0 statevector simulation, 1 Aer noise model, 2 IBM QPU) and `NStates` $= 2^N$. New outputs: `PSubset[NStates]` (the probability of every item subset, indexed by bitmask; for path 0 the exact output distribution of the final circuit), `OracleCalls` (path 0), `Shots`

(paths 1/2) and `Kind` (0 exact distribution of the final circuit, 1 verification shots).

Status codes. Two codes were added to the table in Section 7: `ExStatus` = 1, a solution from a *verification run* on path 1 or 2 (`PatentIndices` valid; the circuit's threshold came from the classical check), and `ExStatus` = −6, an out-of-process path that did not deliver a results file within the timeout.

Solver paths 1 and 2 (out of process). The solver exports the verification circuit — one Grover search at the known optimum value, at the Grover-optimal iteration count — as `solverCircuit.qasm` with a sidecar `solverCircuit.meta` (`N`, `NQ`, `W`, `T`, `j`, `path`, `request_id`, the item qubits, the marked masks), removes any stale results file, and polls every 2 s for up to 2 h (`KNAPSACK_WAIT_TIMEOUT_S` overrides) for `aer_results.json` or `qpu_results.json`: a JSON object with `request_id`, `path`, `shots`, `counts_by_mask` (2^N integers summing to `shots`), `backend` and `job_id`, written to a temporary name and renamed. The file is validated against the request; the most frequent feasible measured subset is the answer. **The repository ships no program that executes the circuit and writes this file**; `hardware_run.py` (Section 10.5) is a demonstration of Qiskit Runtime submission and does not implement the protocol.

The Modelica side. `ModKnapsack.mo` is now a package: a `TestCase` record (N, costs, values, default budget and solver path), a `TestCases` library of twenty instances (the four-item golden case of Section 8, the two-item hardware case, and the families TC1 loose budget, TC2 tight budget, TC3 unique optimum at $N = 2 \dots 7$), the external function `ModCallKnapsack`, and the model `KnapsackRun`, which selects a case, lets the user override `MaxCost` and `SolverPath`, calls the solver once at initialisation and prints the result, the most probable subsets and the blind-draw chance $1/2^N$.

Limits. `KNAPSACK_NMAX` is **7** (this document says 4): the statevector reaches 21 qubits at $N = 7$. The ancilla-assisted $C^{N-1}Z$ in the diffuser, listed under "genuinely remaining" in Section 11, is implemented: for $N \geq 5$ the multi-controlled Z is built from a Toffoli ladder into clean cost/value/carry qubits, a CCZ, and the ladder uncomputed, so the Qiskit C API's native gate set (`CCX`, `CCZ`, `C3X`) suffices at every N. `GROVER_K` is still 3. The seven-item solves take minutes each on a desktop.

Tests and build. The white-box programs are the nine of Section 9; `diffuser_test` now checks the reflection identity for every $N \leq 7$ and one-iteration amplification at $N = 5$ and $N = 7$, `solver_test` includes a five-item search, and `knapsack_test` exits non-zero on any failure. The build runs from the repository root (the directory names `Experiment08/Experiment09` in Sections 5 and 9 are the author's working trees); "Tier-1" (this document's title and running head until this revision), "Tier-2" (Section 10) and the "Stage" column of Table 2 are the author's development milestones: the simulator-based solver, the export/noise/hardware tools, and the order in which the sections were built and tested. The per-stage snapshots, the author's development notes and the reader's guide are internal records and are **not** in the public repository; the documents that are published are listed at the end of this preface.

Results. The install-gate case ($N = 4$, items 0 and 2, cost 6, value 8, 59 oracle calls) reproduces. Across the 90 library-style instances used for the conference talk (TC1/TC2/TC3, $N = 2 \dots 7$, five instances each) the solver returned the optimum on every one; the mean oracle-call count grows roughly as $2^{0.7N}$ and exceeds the 2^N brute-force evaluations at every N — no quantum advantage is claimed.

Companion repository documents: `README.pdf` (what the repository is, how the Modelica model calls the solver, the three solver paths, build and test), `ALGORITHM_REFERENCES.pdf` (annotated bibliography), `ModKnapsack_Installation_Instructions.pdf` (installation and the install gate) and `ModKnapsack_Modelica2026-Paper.pdf` (the accepted conference paper). The author's development notes and the reader's guide `ModKnapsack_Algorithms` are not in the public repository. Readers new to quantum computing should start with the conference paper before this document's full engineering treatment.

1 Introduction and Scope

The classical 0/1 knapsack problem selects a subset of N items, each with a cost c_i and value v_i, that maximises the total value subject to a budget W:

$$\max_{x\in\{0,1\}^N} \sum_{i=0}^{N-1} v_i x_i \quad \text{subject to} \quad \sum_{i=0}^{N-1} c_i x_i \;\le\; W. \tag{1}$$

Here $x_i = 1$ means "item i is selected." The software solves (1) by moving the *selection* into a quantum circuit: an oracle phase-marks the feasible, high-value subsets, a diffuser amplifies them, and an outer loop drives a value threshold up to the exact optimum.

Purpose of this document. This is detailed software documentation for the author and for public release on GitHub alongside the code. It is deliberately distinct from the 2026 American Modelica & FMI Conference paper: it favours completeness and reproducibility over narrative.

Tier roadmap. Development is organised into tiers. **Tier-1** (this document) delivers a complete, tested, end-to-end quantum solver on an ideal statevector simulator. Future tiers add noise modelling, real-hardware execution, and a publication-grade circuit-drawing capability (Section 11).

Note

Honest framing is a hard requirement (Section 2.3). On a statevector simulator there is no quantum speedup. The contribution is a genuine quantum *algorithm* executed for validation, with the standard quadratic Grover envelope of $O(2^{N/2})$ oracle calls.

2 Requirements

2.1 Functional requirements

ID	Title	Requirement
FR1	Solve knapsack	Return an optimal feasible subset for (1).
FR2	Quantum selection	The subset is selected by a quantum algorithm (Grover amplitude amplification), *not* by classical brute force.
FR3	Full predicate	Both constraints — budget $\sum c_i x_i \le W$ *and* a value threshold $\sum v_i x_i \ge T$ — are evaluated *in the circuit* (Tier-1 scope).
FR4	Exact optimum	Return the exact maximum-value feasible subset (Dürr–Høyer maximum finding).
FR5	OpenModelica	Callable from OpenModelica as an external C function.
FR6	Circuit output	Optionally emit a diagram of the quantum circuit.
FR7	Statistics	Report solver statistics (oracle-call count, measured-outcome histogram) for post-processing.

2.2 Non-functional requirements and contracts

ID	Title	Requirement
NFR1	OpenModelica contract	`callKnapsack()` returns `void` with all outputs by pointer; it is the *only* external-linkage symbol (all else `static`); the file is one translation unit (`#include`d by OpenModelica); the build is warning-clean; `ActualCost` is computed in Modelica, not C.
NFR2	Vanilla Qiskit	No edits to Qiskit source, makefiles, or the C API; only the pre-built `libqiskit.so` and headers under `qiskit/dist/c` are consumed.
NFR3	Pure quantum	The answer comes only from the quantum search; there is no classical fallback. Problem size is gated by `KNAPSACK_NMAX`; larger inputs return status -3 ("too large").
NFR4	Reproducibility	A deterministic per-instance seed makes each run repeatable (PRNG: xorshift64* [15]).
NFR5	Self-contained	The statevector simulator is pure C; no external simulation or linear-algebra libraries are used.

2.3 Correctness and honesty rules (hard requirements)

These are the correctness and honesty constraints the implementation is held to:

IR1. **Oracle blind to the answer.** The threshold T comes only from the Dürr–Høyer loop. The oracle computes cost and value *from the item qubits* and is never handed the classical optimum.

IR2. **Uncomputation mandatory.** All ancilla registers return to $|0\cdots0\rangle$ before the diffuser — otherwise interference is wrong. The technique is Bennett's [12], textbook form in [9, §3.2.5].

IR3. **No speedup claims.** Honest framing only (Section 6.8).

IR4. **No silent skips.** Every emitted gate must have a simulator handler; the walker aborts loudly on any unhandled gate.

2.4 Updated requirement (logged 2026-06-05)

Circuit-drawing detail. A `QDrawDetail` level (0–5) replaces the binary print flag, with levels > 3 producing a publication-quality black-and-white figure in the standard (Nielsen & Chuang) gate symbols, generated *off-line* from an exported circuit description rather than hand-drawn in C. *Implemented* in Section 10.4.

3 Fundamental Publications

The implementation is built directly from the works in Table 1. Each section of `knapsack.c` carries an inline "References" note tying its code to these entries. Full annotations are in `ALGORITHM_REFERENCES.pdf`.

Warning

The arXiv identifiers and years below are recorded from the implementer's knowledge and **must be verified against the primary sources before they appear in the conference paper**. They are adequate for this internal/engineering document.

Ref	Work	Contribution used	Used in
[1]	Grover 1996	amplitude amplification (oracle + diffuser)	§2.7,§2.8,§2.9
[2]	Boyer–Brassard–Høyer–Tapp 1998	unknown-M iteration schedule	§2.9
[3]	Brassard–Høyer–Mosca–Tapp 2002	amplitude-amplification framework	§2.8
[4]	Dürr–Høyer 1996	quantum minimum/maximum finding	§2.9
[5]	Vedral–Barenco–Ekert 1996	reversible controlled adders	§2.5
[6]	Cuccaro et al. 2004	ripple-carry adder; carry-out = sign bit	§2.5,§2.6
[7]	Draper 2000	QFT adder (alternative, *not* used)	§2.5
[8]	Barenco et al. 1995	multi-controlled gate decomposition	§2.8 (ext.)
[9]	Nielsen & Chuang 2010	Grover oracle; "inversion about the mean"	§2.7,§2.8
[10]	Wilkening et al. 2025	Quantum Tree Generator (alternative, not used)	—
[11]	Farhi et al. 2014	QAOA ("Option B," not chosen)	—
[12]	Bennett 1973	logical reversibility; uncomputation	IR2, §2.7
[13]	Bennett et al. 1997	optimality of the quadratic envelope	§6.8
[14]	Cross et al. 2022	OpenQASM 3 language and `stdgates.inc`	§4.5
[15]	Vigna 2016	xorshift64* PRNG	§2.9
[16]	Javadi-Abhari et al. 2024	Qiskit architecture (the C API's host project)	§4

Table 1: Literature map: publications and where each is used.

Design choices relative to the literature. We use the integer-exact ripple-carry adder [5, 6] rather than the QFT phase adder [7, §§3–4] (readability and exactness); Grover maximum-finding [1, 4] rather than the approximate QAOA [11]; and a plain feasibility oracle rather than the Quantum Tree Generator [10] (easier to validate from scratch).

4 Architecture

Figure 1 shows the end-to-end pipeline. OpenModelica passes the problem to a single external C function. The C code builds a quantum circuit with the Qiskit C API, evaluates it on a pure-C statevector simulator, runs the Grover/BBHT/Dürr–Høyer search, and returns the selected subset.

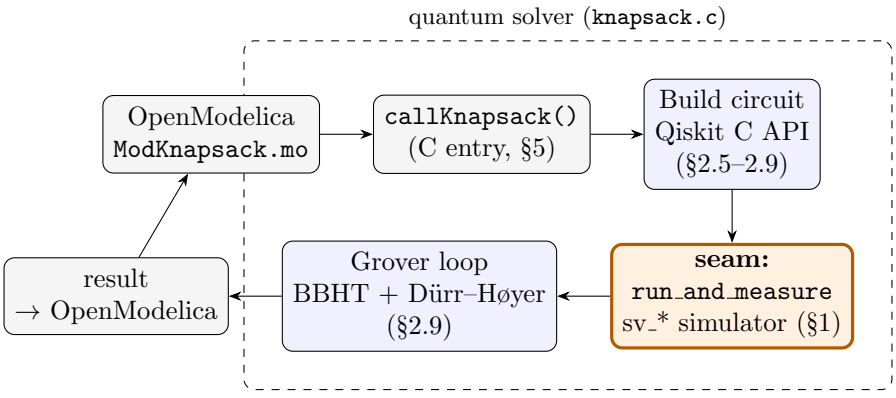


Figure 1: End-to-end architecture. The orange block is the single "execution seam": replacing `run_and_measure` (today the ideal simulator) with a noisy simulator or real hardware is the entire surface for future tiers.

4.1 Layering and the execution seam

The design separates *circuit construction* from *circuit execution*:

- `build_grover_circuit()` constructs the gate sequence (hardware-faithful; only real Qiskit gates). This is the artifact a noisy simulator or a QPU would consume.
- `run_and_measure()` executes the circuit and samples one outcome. Today this is the pure-C statevector simulator, and it is the *single function* that a future tier swaps to introduce noise (Aer) or hardware (Qiskit Runtime), leaving everything above it unchanged.

4.2 The role of the Qiskit C API

The Qiskit C API [16] is a *circuit-construction and transpilation* interface only — it exposes no simulator, no noise model, and no hardware client (verified against the headers). Consequently the statevector simulator (Section 7.1) was written from scratch. It is not a reinvention but the only way to evaluate a circuit within the pure-C / OpenModelica constraints. The bridge to the Python ecosystem (for future noise and hardware work) is `qk_circuit_to_python` / OpenQASM export.

4.3 Pure-quantum scope gate

The simulator stores 2^{N_Q} complex amplitudes, where $N_Q = N + m_{\text{cost}} + m_{\text{value}} + m_{\text{carry}} + 2$. The constant `KNAPSACK_NMAX` (default 4, giving $N_Q = 20$) caps the accepted problem size; larger inputs return status -3. There is deliberately *no* classical fallback: the contribution is a quantum solver, and `KNAPSACK_NMAX` is raised as simulator or hardware capacity grows.

5 Code Structure

The solver is one C file, `knapsack/knapsack.c`, organised into the sections of Table 2; the OpenModelica model is `ModKnapsack.mo`; the white-box tests each `#include "knapsack.c"` (the same inclusion model OpenModelica uses) so they can reach the `static` helpers.

Section	Contents	Key symbols	Stage
§1	pure-C statevector simulator	`sv_h/x/y/rz/cx/mcx/mcz`, `sv_alloc`	1
§2	circuit walker / dispatch	`simulate_circuit`	1
§2.5	controlled constant-adder	`emit_controlled_const_add`, `emit_carry`	2
§2.6	comparator / feasibility flag	`emit_flag_leq`, `emit_flag_geq`	3
§2.7	Grover oracle	`emit_oracle`, `emit_controlled_const_sub`	4
§2.8	Grover diffuser	`emit_diffuser`, `emit_cnz`	5
§2.9	BBHT + Dürr–Høyer solver	`grover_knapsack`, `grover_search`, `durr_hoyer`	6
§3	solver wiring	`solve_knapsack`	7
§4	LaTeX circuit figure	`write_latex_circuit`	7
§5	OpenModelica entry	`callKnapsack`	7

Table 2: Structure of `knapsack.c`.

Build targets. `CMakeLists.txt` declares the `knapsack` shared library (loaded by OpenModelica) plus seven test executables (one per stage). Snapshots `Version-0` through `Version-6` are per-stage rewind points.

Key data structures. The solver passes an instance/layout context and an optional statistics record:

```
typedef struct { /* gctx: instance + qubit layout */
  int N; /* number of items */
  const uint32_t *costs, *values;
  uint32_t W; /* budget */
  uint32_t m_cost, m_val, m_carry; /* register widths */
  int NQ; /* total qubits */
  uint32_t item[32], cost_q[32], value_q[32], carry_q[32], feas, valok;
} gctx;

typedef struct { /* gstats: solver statistics (optional) */
  int K; /* best-of-K runs */
  uint32_t run_value[32]; long run_oracle_calls[32];
  uint32_t best_value, best_mask;
  long total_oracle_calls; /* the Grover query-complexity metric */
  int total_grover_samples, total_dh_rounds;
  long measure_hist[256]; /* measured-outcome histogram */
} gstats;
```

6 Algorithms

6.1 The quantum predicate

A subset x is *marked* by the oracle iff it is feasible and meets the current value threshold:

$$\text{MARK}(x) \;=\; \Big[\textstyle\sum_i c_i x_i \le W\Big] \;\wedge\; \Big[\textstyle\sum_i v_i x_i \ge T\Big]. \tag{2}$$

6.2 Reversible arithmetic: controlled constant-adder (§2.5)

For each item i, controlled on the item qubit, the constant c_i is added into the cost register by a ripple-carry network of $X/CX/CCX/C3X$ gates [5, Figs. 2–3]. The carry out of bit j is the majority $\text{MAJ}(a_j, \text{reg}_j, \text{carry}_{j-1})$ — Cuccaro et al.'s carry recursion $c_{i+1} = \text{MAJ}(a_i, b_i, c_i)$ [6, §2, Fig. 1] — and gating the whole network on an item qubit is exactly the conditional addition of [5, Fig. 5]. The carry scratch is *fully uncomputed*: a forward pass computes all carries (register read-only), then a top-down pass uncomputes each carry *before* writing the sum bit at that position — avoiding the hazard where the sum overwrites a carry's input. Net effect: cost $+= c_i \pmod{2^m}$, carry scratch back to $|0\rangle$.

Worked micro-example. Add the constant $a = 3$ (011_2) into a 3-bit register holding 2 (010_2), control qubit $= |1\rangle$. Bit by bit (sum $= a_i \oplus \text{reg}_i \oplus$ carry-in, carry-out = majority of the three): bit 0 has $(a_0, \text{reg}_0, \text{cin}) = (1, 0, 0)$, so sum bit 1 and carry-out 0. Bit 1 has $(1, 1, 0)$, so sum bit 0 and carry-out 1. Bit 2 has $(0, 0, 1)$, so sum bit 1 (the MSB produces no carry-out — the register is sized so none is needed). The register now reads $101_2 = 5 = 2 + 3$, and the backward pass has already returned both scratch carries to $|0\rangle$. When the control qubit is $|0\rangle$ every gate is suppressed and the register is untouched — which is exactly how, run on a superposed control, the same gate sequence adds c_i *only* on the branches that include item i.

6.3 Comparator via two's complement (§2.6)

To test reg $\le W$ over m bits, add the constant $K = (2^m - (W+1)) \bmod 2^m$ and read the carry-out of the most-significant bit: it equals the truth of reg $\ge W + 1$, so $\neg$(carry-out) is feasibility. The adder-into-comparator modification is noted by Cuccaro et al. themselves [6, §1]. The comparator needs only the carry network (no sum), so it runs a forward carry pass,

copies the sign bit into a flag qubit, then a reverse pass restores the carries — leaving the register untouched. The value test reg $\geq T$ uses $K = (2^m - T) \bmod 2^m$ and reads the carry-out directly.

Worked micro-example (the same one annotated in `knapsack.c`). Over $m = 5$ bits ($2^m = 32$), test cost ≤ 14: $K = 32 - 15 = 17$. A register holding 14 gives $14 + 17 = 31$, no carry out of bit 4, so the subset is feasible. A register holding 15 gives $15 + 17 = 32$, carry out $= 1$, infeasible. For value ≥ 14: $K = 32 - 14 = 18$, and $13 + 18 = 31$ (no carry, below threshold) versus $14 + 18 = 32$ (carry, threshold met). One addition-without-sum per predicate, and the flag qubit carries the verdict.

6.4 Grover oracle: compute → mark → uncompute (§2.7)

The oracle U_f phase-flips exactly the marked states and returns all ancillas to $|0\rangle$ [9, §§3.2.5, 6.1.2]:

1. Accumulate $\sum c_i x_i$ and $\sum v_i x_i$ into ancilla registers (per-item controlled adds, §2.5).
2. Derive the two predicate flags (§2.6).
3. **Mark:** a single controlled-Z on the two flag qubits applies -1 to the branch where both hold (feasible $\wedge$ value-ok).
4. **Uncompute** (a mirror image of steps 1–2). The comparators are self-inverse (call again), while the adder is *not* (adding k twice adds $2k$), so each accumulation is reversed by adding the two's complement $2^m - k$.

Because every block maps a basis state to a single basis state, the diagonal mark factors through the uncompute, leaving $(-1)^{\mathrm{MARK}(x)}$ on the item register alone (IR1, IR2).

6.5 Grover diffuser (§2.8)

The diffuser reflects about the uniform state $|s\rangle = H^{\otimes N}|0\rangle$ — Grover's "diffusion transform," step (b) of his algorithm [1], named the "inversion about mean" by [9, §6.1.2], and the special case of amplitude amplification with $\mathcal{A} = H^{\otimes N}$ [3, §2, Thms. 2–3]:

$$H^{\otimes N}\, X^{\otimes N}\, C^{N-1}Z\, X^{\otimes N}\, H^{\otimes N} \;=\; I - 2|s\rangle\langle s|,$$

acting on the item register alone (the oracle leaves ancillas clean). For $N = 4$ the core C^3Z is realised as $H \cdot C3X \cdot H$ on one qubit (native gates top out at three controls). One Grover iteration takes the marked-state amplitude from $\sin\theta$ to $\sin 3\theta$, with $\sin\theta = \sqrt{M/2^N}$ — each iteration is a rotation by 2θ in the marked/unmarked plane [9, §6.1.3].

6.6 Inner search: BBHT (§2.9)

When the number of marked states M is unknown, a fixed iteration count fails. BBHT [2] uses an exponential schedule — their main algorithm, steps 1–6, with $\lambda = 6/5$ the published choice and "any value of λ strictly between 1 and 4/3" admissible: $m \leftarrow 1$, grow by λ (capped at $\sqrt{2^N}$). Each round pick a random $j \in [0, \lceil m \rceil)$, apply j Grover iterations to $|s\rangle$, measure, and accept the sample only if it is genuinely marked. If $M = 0$ the oracle is the identity and no sample is ever marked, so the search reports failure after its budget. The success guarantee (probability $\geq 1/4$ once m exceeds $1/\sin 2\theta$) is their Lemma 2, and the expected cost is their Theorem 3.

6.7 Outer loop: Dürr–Høyer maximum finding (§2.9)

The outer loop is Dürr–Høyer's threshold algorithm [4, their Algorithm and Thm. 1]. The best feasible solution starts as the empty set (value 0). Set $T = \text{best}+1$ and run the inner search. On success raise best and set $T = \text{value}+1$, while on failure stop — best is the optimum. Crucially T is raised *only* from measured results (IR1). The solver runs this whole loop K times (best-of-K) and returns the highest value found, a reliability margin that becomes important under noise.

Instrumented trace (the N=4 baseline, deterministic seed, golden re-run of 2026-07-12). The three best-of-K runs each found the optimum value 8, spending 17, 24, and 18 oracle calls respectively — 59 in total across 11 Dürr–Høyer rounds and 64 circuit samples. `groverStats.txt` reproduces this table on every run, and the oracle-call column is the honest cost metric of Section 6.8.

6.8 Note on computational complexity

Grover concentrates amplitude on the M marked states in $\Theta\big((\pi/4)\sqrt{2^N/M}\big)$ oracle calls [2, Thm. 3][9, §6.1.4], while Dürr–Høyer reaches the optimum in $O(\sqrt{2^N})$ calls overall [4, Thm. 1]. On a statevector simulator there is no wall-clock speedup — simulating each oracle call is itself exponential. The defensible, reported metric is therefore the *oracle-call count* (in `groverStats.txt`), the standard quadratic Grover query envelope — which is, moreover, provably the best possible for unstructured search [13, §3.1, Thm. 3.5]. **No speedup over classical solving is claimed** (IR3).

7 Implementation Details

7.1 Pure-C statevector simulator (§1)

A complex array of 2^n amplitudes is updated in place, with qubit q occupying bit q of the basis index (qubit 0 is least significant). The permutation gates (X, CX, CCX, $C3X$ via `sv_mcx`) and the phase flips (`sv_mcz`) are *exact* — they only permute or sign-flip amplitudes, with no floating-point rotation. `simulate_circuit` reads each instruction back from the circuit and dispatches on its *name string*.

Warning

Dispatch is by canonicalised name, not enum. The Qiskit C API returns a gate's name string on read-back, and it is *not* the enum identifier: `QkGate_C3X` reads back as `"mcx"` (while `QkGate_CCX` reads back as `"ccx"`). The dispatch must use the probed names. A **silent-skip guard** (IR4) aborts loudly on any gate with no handler, so a missing handler can never corrupt the statevector quietly.

Warning

`sv_phase` is the Phase gate, not RZ. `sv_phase` applies $\text{diag}(1, e^{i\theta}) = P(\theta)$, which is *exact* for the gates that use it ($Z, S, S^\dagger, T, T^\dagger$, Phase). The "rz" dispatch maps true $RZ(\theta) = \text{diag}(e^{-i\theta/2}, e^{+i\theta/2})$ onto $P(\theta)$. The two differ by a global phase $e^{i\theta/2}$, unobservable for an isolated gate [9, §2.2.7], and the Grover circuit emits no `rz`/`phase` at all. It would matter only under a *controlled* `rz`, which is never used. (Renamed from `sv_rz` to `sv_phase` in v1.3; if true RZ is ever required, add a separate $\text{diag}(e^{-i\theta/2}, e^{+i\theta/2})$ handler.)

7.2 Qubit layout

Registers are laid out contiguously and sized so the accumulators never overflow ($2^{m_{\text{cost}}} > \sum c_i$, $2^{m_{\text{value}}} > \sum v_i$):

$$\underbrace{\text{item}}_{N} \mid \underbrace{\text{cost}}_{m_{\text{cost}}} \mid \underbrace{\text{value}}_{m_{\text{value}}} \mid \underbrace{\text{carry}}_{m_{\text{carry}}} \mid \text{feas} \mid \text{valok}.$$

For the $N = 4$ baseline ($c = [2, 3, 4, 5]$, $v = [3, 4, 5, 7]$) this is $4 + 4 + 5 + 5 + 2 = 20$ qubits.

7.3 Solver wiring (§3)

`solve_knapsack` converts the model's (small-integer) doubles to `uint32_t`, clamps the budget into the cost register's range, derives a deterministic per-instance seed, runs `grover_knapsack` (the quantum selection), and *independently* computes the classical optimum as a cross-check (logging on any disagreement but never overriding — IR1). It writes `groverStats.txt`, maps the result bitmask to `PatentIndices`, and returns the value or a status sentinel.

Warning

Integer arithmetic. The quantum cost/value registers are integers, so non-integer `Costs`/`Values` are rounded to the nearest integer (the model's contract is small integers). For fractional inputs the selected subset may differ from the real-valued optimum.

7.4 OpenModelica entry (§5) and status codes

`callKnapsack` is `void` with all outputs by pointer (mixing a C return with pointer outputs corrupts OpenModelica's stack). `ActualCost` is computed in Modelica. Status codes:

ExStatus	Meaning
0	solution found; `PatentIndices` valid
−1	no feasible solution (only the empty set fits)
−2	invalid input (out of range or allocation failure)
−3	problem too large for the quantum solver ($N >$ `KNAPSACK_NMAX`)

7.5 Compile-time knobs

`KNAPSACK_NMAX` (largest accepted N, default 4), `GROVER_K` (best-of-K, default 3), `WRITE_GROVER_STATS` (emit the stats file). All are overridable at compile time, e.g. `-DGROVER_K=5`.

8 Verification and Test Results

8.1 Strategy

Each stage was *derived, then tested* before the next was built: hand-derive the circuit block, check small cases by hand, then confirm with a white-box test that drives the actual emitted gates through the real simulator. The strongest test is the **oracle self-check** (Stage 4): the oracle is simulated on the uniform superposition and asserted to phase-mark *exactly* the states an independent classical pass finds feasible-and-high-value — the same set derived two independent ways, which simultaneously proves IR1.

8.2 Results

All suites pass and the build is warning-clean (Table 3).

Suite	Scope	Result
`gate_test`	§1 simulator gates (incl. `sv_mcx`/`sv_mcz`)	6/6
`adder_test`	§2.5 controlled adder (7 primitives + all 16 subsets)	23/23
`cmp_test`	§2.6 comparator (boundary + exhaustive sweeps, 448 cmp)	19/19
`oracle_test`	§2.7 oracle self-check (1 hand case + 4 N=4 cases)	all pass
`diffuser_test`	§2.8 reflection identities + one-iteration amplification	all pass
`solver_test`	§2.9 BBHT+Dürr–Høyer vs classical optimum	N=3: 20/20; N=4: pass
`knapsack_test`	§3–5 end-to-end (TC1–TC7, incl. N=6 $\rightarrow$ −3)	7/7
OpenModelica	`omc` compile + simulate `KnapsackRun`	pass

Table 3: Tier-1 verification results.

Quantitative checks. The diffuser test confirms one Grover iteration drives a single marked state from $1/16 = 6.25\%$ to $\sin^2(3\theta) = 47.27\%$, matching theory to four decimals. The solver reports the Grover oracle-call count, e.g. ≈ 59 for the $N = 4$ baseline.

End-to-end result. Running the OpenModelica model (`simulate(ModKnapsack.KnapsackRun)`) on the baseline instance with budget 6 returns `ExStatus`= 0, `PatentIndices`= $[1, 0, 1, 0]$ (items 0 and 2), `ActualCost`= 6, quantum value 8 — the exact optimum, selected by Grover, agreeing with the classical checker.

9 Build and Run

From the project root — the tree that contains `knapsack/`, `qiskit/`, and `ModKnapsack.mo` (currently `Experiment09/`) — with the prebuilt `libqiskit.so` on the runtime library path:

```
cmake --build cmake-build-debug
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/gate_test # 6/6
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/adder_test # 23/23
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/cmp_test # 19/19
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/oracle_test # self-check
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/diffuser_test # reflection+iter
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/solver_test # BBHT+DH (~50s)
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/knapsack_test # 7/7 end-to-end
LD_LIBRARY_PATH=qiskit/dist/c/lib omc run_knapsack.mos # OpenModelica
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/qasm_test # 9/9 export
LD_LIBRARY_PATH=qiskit/dist/c/lib ./cmake-build-debug/draw_test # 15/15 QDrawDetail
# Out-of-band consumers of the exported circuit (project .venv):
python3 verify_qasm.py # Qiskit round-trip, fidelity 1.0
python3 noise_study.py # Aer depolarizing sweep -> noise_study.csv
python3 plot_noise.py # -> noise_study.png/pdf (depth-scaling figure)
python3 draw_circuit.py --qasm qasm_xcheck/grover.qasm --pdf # B&W N&C figure
python3 hardware_run.py # qiskit-ibm-runtime SamplerV2 (FakeBackend; real QPU needs token)
```

10 Tier-2: Portable Export and Noise Characterization

Tier-2 adds the capabilities the execution seam (Section 4) was designed to enable, *without* disturbing the pure-C / OpenModelica path: a portable circuit export and an out-of-band noise study built on it. The C solver continues to run on its own `sv_*` simulator; Qiskit and Aer only ever *consume* the exported circuit, so the solver stays purely quantum and the OpenModelica contract is untouched.

10.1 OpenQASM-3 exporter (§4.5)

`write_qasm()` serializes a built `QkCircuit` to OpenQASM 3.0 [14] by walking the instruction list with the *same* strict name-dispatch and silent-skip guard as the simulator (Section 7.1). Standard gates (`h, x, y, z, cx, cz, ccx`, ...) map to `stdgates.inc`, the standard library the OpenQASM 3 specification defines [14]; the two non-standard gates this project emits — `ccz` and the multi-controlled `mcx` (C3X) — are written, exactly as Qiskit's own exporter does, as one-time `gate` definitions (ccz $= H \cdot \mathrm{CCX} \cdot H$, via the conjugation identity $HXH = Z$ [9, §4.2]; `mcx` the standard $p(\pm\pi/8)$-phase–CX ladder, Qiskit's canonical C3X decomposition, rooted in the multi-controlled constructions of [8, §7]), emitted only when used. The exporter is pure C with no embedded Python, so one source feeds all three downstream consumers: Aer (noise), Qiskit Runtime (hardware), and the `QDrawDetail` renderer.

Cross-validation (fidelity 1.0). `verify_qasm.py` re-loads each exported circuit into Qiskit, computes its statevector, and compares it (global-phase-tolerant fidelity) to the `sv_*` statevector. On both a kitchen-sink circuit and the real 124-gate N=2 Grover circuit the fidelity is 1.000000000000 — which simultaneously confirms the exporter *and* independently cross-validates the entire pure-C simulator against Qiskit, a bonus Tier-1 confidence result. The C-side test `qasm_test` (9/9) builds the circuits and writes the artifacts; the validation runs out-of-band in the project `.venv`.

10.2 Aer depolarizing noise study

`noise_study.py` loads an exported Grover circuit into Qiskit Aer, applies an all-qubit depolarizing model (two-qubit error p_2, one-qubit $p_1 = p_2/10$), and measures P(success) — the probability that the measured item register is the marked (optimal-feasible) subset. The C side builds each study circuit at a known optimum and writes a metadata sidecar (measured qubits + the classically-computed marked set via the solver's own `is_marked`), so the ideal success is high by construction and Python performs only the quantum sampling. Two instances were swept (Table 4): a clean N=2, M=1 circuit (356 two-qubit gates after transpilation to $\{u, \mathrm{cx}\}$) and the full N=4 baseline (NQ=20, 6048 two-qubit gates).

p_2 **(two-qubit)**	P**(success),** N=2	P**(success),** N=4
0 (ideal)	1.000	0.970
10^{-5}	—	0.920
10^{-4}	0.975	0.643
10^{-3}	0.786	0.100
3×10^{-3}	0.512	0.078
10^{-2}	0.275	—
10^{-1}	0.248 ($\to 1/4$)	—

Table 4: Depolarizing-noise degradation of P(success). Each curve decays monotonically to the fully-mixed floor $1/2^N$ (a correctness check). Dashes are levels not swept for that instance. Full data in `qasm_xcheck/noise_study.csv`.

Depth-scaling law. The noise-tolerance "knee" (where P falls to half of ideal) sits at $p_2 \approx 3 \times 10^{-3}$ for N=2 but at $p_2 \approx 1.7 \times 10^{-4}$ for N=4. The two circuits differ by $6048/356 \approx 17\times$ in two-qubit gate count, and the knee shifts by the same $\approx 17\times$: noise tolerance scales as $1/n_{2q}$, exactly as expected once the accumulated error $\sim p_2\, n_{2q}$ reaches order unity.

Practical consequence. At present-day superconducting two-qubit error rates ($\sim 10^{-3}$) the small N=2 circuit is marginal ($P \approx 0.79$) but the full-size N=4 oracle is destroyed ($P \approx 0.10$,

near the 1/16 random floor). The solver's best-of-K wrapper extends the usable range (e.g. 0.64 → 0.95 at $p_2 = 10^{-4}$, K=3) but cannot rescue the deep-oracle regime. This quantifies the gap between algorithmic correctness in simulation and NISQ executability, and motivates shallower oracle synthesis as the primary lever. No quantum speedup is claimed (IR3).

10.3 Noise-study figure

`plot_noise.py` renders `noise_study.csv` to a publication figure (P(success) versus p_2 on a log axis, with the best-of-K companion and the $1/2^N$ asymptote): the N=4 knee sits $\approx 17\times$ left of N=2, making the depth-scaling law visual. The device-calibrated-noise extension is covered by `hardware_run.py` below (a `FakeBackend` is a `NoiseModel.from_backend`-equivalent calibrated device model).

10.4 Publication circuit drawing (`QDrawDetail`, §4.6)

`callKnapsack`'s last argument is reinterpreted from the binary `PrintCircuit` flag into a detail *level* `QDrawDetail`$\in \{0\,..\,5\}$ — the same frozen `int`, so the OpenModelica signature is unchanged (the Modelica-side parameter is renamed to match). The dispatcher `write_circuit_drawing()` maps: 0 none; 1 gate-level text via the C API `qk_circuit_draw`; 2 the light schematic (Appendix B); 3 an expanded register-level schematic (`write_latex_circuit_expanded`: every item/cost/value/carry/flag wire, with the oracle as a labelled pipeline $+c{\cdot}x$ | $+v{\cdot}x$ | $\leq W$ | $\geq T$ | mark | uncompute, then D); and 4/5 a full gate-level OpenQASM-3 export of one iteration / the full single-target depth $\lceil(\pi/4)\sqrt{2^N}\rceil$ [9, §6.1.4]. Levels 4/5 feed the off-line renderer `draw_circuit.py` (OpenQASM-3 → Qiskit `latex_source` = qcircuit / Nielsen & Chuang [9] → `pdflatex`), so the C side stays Python-free. The drawn circuit uses an *answer-blind* threshold T=1 (the schematic structure is T-independent, and while the comparator's emitted gate list varies with the bit pattern of its compare constant, the drawn circuit is exactly the T=1 circuit it advertises — faithful, yet unable to encode the optimum — IR1). `draw_test` (15/15) checks every level; the expanded schematic and an N=2 gate-level render both compile to clean B&W figures, and the OpenModelica end-to-end run with `-override=QDrawDetail=3` fires the dispatcher and still returns the optimum. A shared `setup_gctx()` now lays out the registers for the solver, the drawing path, and `qasm_test` alike.

10.5 Hardware path (path-preserving)

`hardware_run.py` submits the C-exported OpenQASM-3 through `qiskit-ibm-runtime`'s `SamplerV2` — the same code path a real IBM QPU run takes. It is *path-preserving*: `--backend fake`/`aer` run locally with no IBM account, while `--backend least-busy`/⟨name⟩ with `--token` reaches real hardware (with a graceful `FakeBackend` fallback). On an ideal simulator it recovers the optimum (P=1.0, confirming the decode); on `FakeBrisbane` the N=2 circuit routes to 825 two-qubit gates and device noise washes it to $P \approx 0.25$ (= 1/4, fully mixed) — consistent with the depth-scaling law, and reported honestly with no advantage claim. The C / OpenModelica path is untouched, and a real-device demonstration awaits an IBM Quantum account.

11 Extension Points and Future Tiers

All remaining work is reached through the execution seam (Figure 1) and the shared OpenQASM-3 exporter, now implemented (Section 10.1). The solver stays purely quantum throughout.

1. **Noise study — completed** (Sections 10.2, 10.3): the out-of-band Aer depolarizing characterization, depth-scaling law, and figure. The device-calibrated comparison is now available through `hardware_run.py` (Section 10.5).

2. **Hardware path — implemented (path-preserving)** (Section 10.5): `hardware_run.py` runs the exported circuit through Qiskit Runtime `SamplerV2` on a `FakeBackend` today; a real-device demonstration awaits an IBM Quantum account. Raise `KNAPSACK_NMAX` as capability grows.

3. **Publication circuit drawing — implemented** (Section 10.4): the `QDrawDetail` 0–5 dispatcher and the off-line `draw_circuit.py` renderer (one representative oracle+diffuser iteration, captioned as repeated under BBHT / Dürr–Høyer).

4. **Cleanups — done.** The `sv_rz→sv_phase` rename is complete (behavior-preserving), and the conference manuscript was revised to reflect the realized future-work items, gained a full-width register-level Grover figure, and had its citations verified against the primary sources.

Genuinely remaining. A real IBM-hardware demonstration (needs an account); shallower oracle synthesis to lift the NISQ depth ceiling; the $N \geq 5$ ancilla-assisted $C^{N-1}Z$ diffuser [8]; and scaling toward industrial portfolio sizes.

A Verified Qiskit C API gate names

Probed against `libqiskit.so`; dispatch must use these strings, not the enum names.

QkGate	name	qubits	order
H / X / Y / Z	h / x / y / z	1	—
Phase / RZ	p (also "phase") / rz	1 (+1 param)	—
CX	cx	2	control, target
CZ	cz	2	symmetric
CCX	ccx	3	c_0, c_1, target
CCZ	ccz	3	symmetric
C3X	**mcx**	4	c_0, c_1, c_2, target

B Circuit schematic (current, light)

Figure 2 is the schematic emitted today (`write_latex_circuit`). It shows the item register only. The cost/value/carry/flag ancillas and the gate-level internals of U_f and D are deferred to the `QDrawDetail`> 3 figure (Section 11).

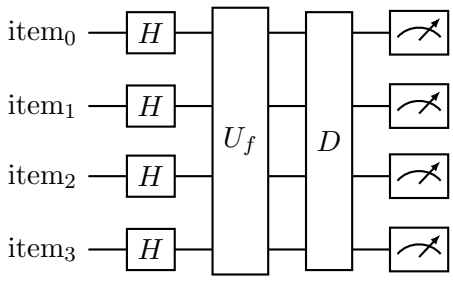


Figure 2: Light Grover schematic: $H^{\otimes N}$ prepares the uniform superposition, then the oracle U_f and diffuser D form one Grover iteration (repeated under BBHT / Dürr–Høyer), then measurement.

Disclosure

During the preparation of this work the author used Anthropic's Claude (LLM chosen appropriate to task and cost, via Claude Code) as a development and editing assistant for the underlying ModKnapsack project — to help resolve build and integration issues with the experimental Qiskit C API and to generate and debug test-harness scripts — and to draft and revise this engineering documentation from the project source code, the test results, and the cited references. All algorithmic design, code, mathematical formulations, and text were directed, independently reviewed, and verified by the author, who retains sole intellectual property ownership of, and full responsibility for, the final work.

References

[1] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proc. 28th ACM STOC*, 1996, pp. 212–219. arXiv:quant-ph/9605043.

[2] M. Boyer, G. Brassard, P. Høyer, A. Tapp, "Tight bounds on quantum searching," *Fortschr. Phys.* 46, 1998, pp. 493–505. arXiv:quant-ph/9605034.

[3] G. Brassard, P. Høyer, M. Mosca, A. Tapp, "Quantum amplitude amplification and estimation," *AMS Contemp. Math.* 305, 2002, pp. 53–74. arXiv:quant-ph/0005055.

[4] C. Dürr, P. Høyer, "A quantum algorithm for finding the minimum," 1996. arXiv:quant-ph/9607014.

[5] V. Vedral, A. Barenco, A. Ekert, "Quantum networks for elementary arithmetic operations," *Phys. Rev. A* 54, 1996, p. 147. arXiv:quant-ph/9511018.

[6] S. A. Cuccaro, T. G. Draper, S. A. Kutin, D. P. Moulton, "A new quantum ripple-carry addition circuit," 2004. arXiv:quant-ph/0410184.

[7] T. G. Draper, "Addition on a quantum computer," 2000. arXiv:quant-ph/0008033.

[8] A. Barenco *et al.*, "Elementary gates for quantum computation," *Phys. Rev. A* 52, 1995, p. 3457. arXiv:quant-ph/9503016.

[9] M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed., Cambridge Univ. Press, 2010 (Ch. 6).

[10] S. Wilkening *et al.*, "A quantum algorithm for solving 0-1 Knapsack problems," *npj Quantum Information* 11, 146, 2025. arXiv:2310.06623.

[11] E. Farhi, J. Goldstone, S. Gutmann, "A quantum approximate optimization algorithm," 2014. arXiv:1411.4028.

[12] C. H. Bennett, "Logical reversibility of computation," *IBM J. Res. Develop.* 17(6), 1973, pp. 525–532.

[13] C. H. Bennett, E. Bernstein, G. Brassard, U. Vazirani, "Strengths and weaknesses of quantum computing," *SIAM J. Comput.* 26(5), 1997. arXiv:quant-ph/9701001.

[14] A. W. Cross *et al.*, "OpenQASM 3: A broader and deeper quantum assembly language," *ACM Trans. Quantum Comput.* 3(3), 2022. arXiv:2104.14722.

[15] S. Vigna, "An experimental exploration of Marsaglia's xorshift generators, scrambled," *ACM Trans. Math. Softw.* 42(4), 2016. arXiv:1402.6246.

[16] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," 2024. arXiv:2405.08810.

Generated as Tier-1 software documentation. The conference paper's citations have since been verified against the primary sources (2026-07-11), while the identifiers here remain as recorded and are re-checked at camera-ready. This document is maintained alongside the code; the "Read first" preface records the October 2026 state.